

研究型专题 A · 第1讲

真实 Agent Benchmark 最小闭环

从 Task、Harness、Tool、Trace、Evaluator 到 Metrics：第一次把 Agent 科研实验真正跑起来

科研洞察 · 这一讲的定位

主课程 52 讲已经解决“知识地图”和“科研方法论”。从现在开始，目标变成：亲手搭出一个可运行、可记录、可评测、可重复的实验系统。本讲故意不追求复杂 Agent，而先把科研基础设施做对。

你将学会什么	本讲结束时能做到什么
Benchmark 的底层结构	明确 Task / Harness / Tool / Trace / Evaluator / Metrics 的职责边界
先跑 Mock，再接 API	先排除 API、网络、SDK 等非科研因素，再接入真实模型
Trace 驱动分析	不只看 accuracy，而是能定位“为什么失败”
论文实验意识	知道怎样从玩具程序升级为可复现实验与论文表格

配套工程：本讲材料包内包含一个完全使用 Python 标准库的最小 Benchmark。默认 Mock 模式可直接运行；OpenAI-compatible API 模式作为后续扩展。

1. 为什么第一步不是“做一个厉害的 Agent”

初学者最容易犯的错误，是一上来就追求复杂规划、长期记忆、多智能体、浏览器工具、几十个 Prompt。这样做看起来“像研究”，但出了问题以后，你往往根本不知道问题在哪一层。

核心概念 · Benchmark 先于复杂 Agent

科研系统最重要的第一步不是让 Agent 更聪明，而是让实验可定义、可重复、可测量、可追踪。如果连“任务是什么、成功是什么、失败发生在哪一步”都说不清，后面增加任何复杂模块都会让因果关系更混乱。

1.1 普通试玩和科研实验有什么差别？

普通试玩	科研 Benchmark
给模型一个问题，看看答得怎么样	预先定义任务集、输入、成功条件与指标
失败后凭感觉改 Prompt	保留 Trace，定位失败环节并形成错误类型
一次运行感觉不错就算成功	多任务、多随机种子、固定配置并可复现
只看最终答案	同时看过程、工具调用、成本、延迟、失败路径
Prompt、模型、数据经常一起变	控制变量，一次只改变研究变量

例子 · 一个最小例子

如果任务是“计算 17×23 ”，最终得到 391 只是结果。科研还会问：Agent 选择了什么工具？参数是否正确？工具结果是否正确？模型是否在最后一步把正确工具结果改错了？这些都需要 Trace 才能判断。

2. 六个部件：Agent Benchmark 的最小闭环



研究真正关心的不只是“最后答对没有”，还包括系统如何走到这个答案。

可以把整个系统想成“实验流水线”。Task 告诉系统要做什么；Agent/Harness 负责运行；Tool/Environment 提供可行动作；Trace 记录过程；Evaluator 判断对错；Metrics 把很多次实验压缩成可比较的数字。

部件	它回答的问题	本讲中的实现
Task	要做什么？什么算正确？	tasks.jsonl
Agent	下一步采取什么行动？	MockAgent / OpenAICompatibleAgent
Harness	如何把一轮实验真正跑起来？	run_one() / main()
Tool / Environment	Agent 可以对外做什么？	calculator / word_count / sort / max / domain

部件	它回答的问题	本讲中的实现
Trace	过程中发生了什么？	agent_action / tool_result / final_answer
Evaluator + Metrics	成功了吗？整体表现如何？	exact_match() + aggregate()

3.Task：不要把“题目”理解成一串自然语言

科研里的 Task 是一个结构化实验单元。它至少应该包含：唯一编号、任务类别、自然语言指令、必要输入、期望输出；复杂研究还会包含难度、扰动类型、数据来源、权限、环境状态等元数据。

```
{
  "task_id": "calc_001",
  "category": "calculator",
  "instruction": "17 * 23",
  "expected": "391"
}
```

核心概念 · 为什么要有 task_id？

因为论文里的任何异常样本都必须能回到原始任务。没有 task_id，你只能说“有一个样本失败了”；有了 task_id，你可以复现实验、打开 Trace、检查 Prompt、对照不同模型。

3.1 为什么使用 JSONL？

JSONL (JSON Lines) 就是“一行一个 JSON 对象”。它非常适合实验数据：便于追加、流式读取、Git diff、逐条检查，也方便以后转换为 Pandas DataFrame。

常见误区 / 风险 · 不要把 expected 当作“模型应该说的完整句子”

评测目标应尽量贴近任务真正关心的语义。本讲为了讲清框架，使用严格 exact match。真实研究中，你可能需要数值容差、集合相等、代码单测、环境状态检查，或独立 Judge。

4. Harness：科研实验真正的“发动机”

Harness 可以理解为“运行框架”或“实验驱动器”。Agent 决定做什么，Harness 决定怎么把一整个实验按固定协议执行。这两者必须分开。

Agent 的职责	Harness 的职责
根据 observation 决定 action	加载任务
选择工具和参数	调用 Agent
根据工具结果生成最终答案	执行工具 / 环境动作
不应该自己偷偷改实验配置	记录 Trace、异常、耗时、配置
不负责整个实验统计	调用 Evaluator 并聚合指标

科研洞察 · 为什么这个区分对论文很重要？

以后你要比较“单 Agent vs 多 Agent”“有记忆 vs 无记忆”“不同通信协议”时，应该只替换研究变量对应的模块，而 Harness 尽量保持不变。否则实验差异可能来自基础设施，而不是你的研究变量。

5. Tool / Environment：Agent 的行动世界

LLM 本身只生成 Token。Agent 之所以成为“系统”，是因为它的输出会被 Harness 解释成动作，再作用于工具或环境。工具可能是计算器、搜索、数据库、Python、浏览器；环境可能是操作系统、游戏、网页、机器人或

模拟器。

```
action = agent.decide(task)
tool_result = TOOLS[action.tool](**action.arguments)
final_answer = agent.finalize(task, tool_result)
```

核心概念 · 底层原理

Agent 输出的 “tool=calculator” 本身不会计算任何东西。真正执行乘法的是 Harness 里的 Python 函数。也就是说：LLM 提议动作，系统执行动作。这正是 Agent 安全、可靠性和系统研究大量问题的来源。

5.1 本讲为什么工具很简单？

因为这里的研究对象不是“工具本身有多强”，而是实验闭环是否正确。工具越简单，我们越容易确认：如果失败，失败到底在 Agent 选择、参数生成、工具执行，还是最终回答。

6. Trace：Agent Systems 科研里最值钱的数据之一

如果只保存最终答案，你得到的是一个“黑盒分数”。如果保存 Trace，你得到的是一条可分析的系统行为链。很多真正有价值的科研现象，恰恰来自 Trace，而不是来自最终 accuracy。

```
[
  {"step":1,"event":"agent_action","tool":"calculator",...},
  {"step":2,"event":"tool_result","result":"391"},
  {"step":3,"event":"final_answer","answer":"391"}
]
```

6.1 Trace 至少应记录什么？

层次	建议字段	研究价值
任务层	task_id / category / difficulty	知道失败发生在哪类任务
模型层	model / temperature / seed / version	控制可复现性
步骤层	step / observation / action / arguments	分析决策链
工具层	tool / result / error / latency	区分模型错误与工具错误
结果层	final_answer / success / error_type	用于统计与错误分析
成本层	tokens / cost / wall time	研究效率与可扩展性

科研洞察 · Trace 不是“日志越多越好”

真正有用的 Trace 应围绕研究问题设计。比如研究通信瓶颈，就要保留每轮消息；研究工具误用，就要保留 tool arguments；研究长期记忆，就要记录 memory read/write。Trace schema 本身就是实验设计的一部分。

7. Evaluator：把“好不好”变成可执行的判定规则

Evaluator 的关键不是“高级”，而是判定规则与研究问题一致。本讲使用 exact match，因为这些任务有唯一明确答案。

```
def exact_match(pred, expected):
    normalize = lambda s: re.sub(r"\s+", "", str(s)).strip().lower()
    return normalize(pred) == normalize(expected)
```

7.1 常见 Evaluator 类型

类型	适合场景	主要风险
Exact match	分类、结构化答案、确定性结果	对格式很敏感
数值容差	浮点计算、测量值	容差选择影响结论
Unit tests	代码生成、程序修复	测试覆盖不足会产生假阳性
Environment check	网页操作、OS 任务	环境本身可能不稳定
LLM-as-a-Judge	开放式文本、复杂质量判断	Judge 偏差、位置偏差、自偏好、可重复性
Human evaluation	主观质量、安全性、复杂语义	成本高、一致性与盲评设计困难

常见误区 / 风险 · 一个危险误区

不要因为 LLM Judge 使用方便，就把所有任务都交给 Judge。能用确定性规则评测的地方，优先用确定性规则；否则你可能把“被评测系统的随机性”与“Judge 的随机性”叠在一起。

8. Metrics：一个分数永远不够

accuracy 是最直观指标，但 Agent Systems 常常存在“答案正确但过程很差”的情况：调用了 20 次工具才答对、成本极高、依赖异常重试、踩了安全边界等。因此科研里通常需要多维指标。

指标	含义	可能回答的研究问题
Task success / accuracy	任务最终是否完成	系统到底能不能做成事情
Steps / tool calls	完成任务用了多少步	规划效率、冗余行为
Latency	墙钟时间	系统部署效率
Token / cost	模型调用成本	扩展性与经济性
Error type distribution	不同失败机制占比	方法究竟解决了哪类问题
Variance across seeds	随机重复之间的波动	稳定性与可重复性

科研洞察 · 论文里的“提升 2%”不一定最重要

如果方法 accuracy 只提高一点，但把“工具选择错误”降低一半、把方差显著压低，可能更有系统科学价值。关键是先提出机制问题，再选能检验机制的指标，而不是先找一个容易涨的总分。

9. 本讲配套工程：目录结构

```
src/  
  tasks.jsonl  
  run_benchmark.py  
  analyze_results.py  
  requirements.txt  
  README.md  
  outputs/  
    results.jsonl  
    traces.jsonl  
    summary.json
```

你现在最重要的不是逐行背代码，而是先建立“文件职责”的脑内地图：任务数据和程序分开；运行和分析分开；原始结果和汇总结果分开。这样以后增加 1000 个任务、5 个模型、10 个随机种子时，工程仍然不会乱。

10. 第一次实验：只跑 Mock Agent

动手做 · 为什么必须先 Mock？

如果一开始就接商业 API，任何失败都可能来自 API key、网络、限流、接口兼容、JSON 格式、模型随机性。Mock Agent 的作用是先证明：Task → Harness → Tool → Trace → Evaluator → Metrics 这条管道是通的。

```
cd src  
python run_benchmark.py --backend mock  
python analyze_results.py
```

本材料在生成时已经实际运行过。当前 8 个示例任务在 Mock 模式下为 8/8 成功，说明基础管道和确定性工具链能够正常工作。注意：这并不说明 Agent 有多强，因为 MockAgent 本质上是规则程序；它只是基础设施自检。

10.1 运行后你应该看到什么？

```
{  
  "n_tasks": 8,  
  "n_success": 8,  
  "accuracy": 1.0,  
  "by_category": {...}  
}
```

常见误区 / 风险 · 如果 Mock 都跑不通怎么办？

此时不要接 API。先修 Harness、任务数据、工具或评测器。科研里这一步非常重要：先把“实验基础设施错误”与“模型行为现象”分离。

11. 如何读 results.jsonl 和 traces.jsonl

一个结果对象大致包含：task_id、category、expected、prediction、success、error_type、latency_ms 和完整 trace。真正研究时，你经常从总表找到异常，再回到某个 task_id 的 trace。

观察	你应该问什么
某一类任务 accuracy 明显低	是工具选择错、参数错、工具失败，还是 finalization 改错？
某些任务成功但 latency 很高	是否出现无效循环、重复调用、上下文膨胀？
同一任务不同 seed 结果波动	系统是否存在脆弱决策点？
多个 Agent 同时犯相同错误	是否存在错误相关性、共享偏差或信息传播问题？
最终答案错但中间 tool_result 对	可能是“正确证据 → 错误整合”的 failure mode

科研洞察 · 这就是从“跑模型”到“研究系统”的转折

如果你只看最终 accuracy，你只知道“失败了多少”；如果你分析 Trace，你开始知道“系统为什么失败、错误在哪里形成、是否能被机制化解释”。后者才更接近 AI Systems Science。

12. 接入真实模型 API：只替换 Agent，不动实验骨架

本工程保留了一个最小 OpenAI-compatible Chat Completions 客户端。它故意不依赖第三方 SDK，这样你能看清最基本的请求/响应过程。不同供应商兼容程度可能不同，因此真正接入时只需要调整 `OpenAIBCompatibleAgent._chat()`，而不是修改 Task、Harness、Trace 或 Evaluator。

```
# Linux / macOS
export AGENT_API_BASE="https://YOUR_PROVIDER/v1"
export AGENT_API_KEY="YOUR_KEY"
export AGENT_MODEL="YOUR_MODEL"
python run_benchmark.py --backend openai-compatible

# Windows PowerShell
$env:AGENT_API_BASE="https://YOUR_PROVIDER/v1"
$env:AGENT_API_KEY="YOUR_KEY"
$env:AGENT_MODEL="YOUR_MODEL"
python run_benchmark.py --backend openai-compatible
```

常见误区 / 风险 · 不要把 API key 写进 Python 文件

实验代码未来很可能上传 GitHub。密钥应通过环境变量或 secret manager 注入。把 key 硬编码进源码，是科研工程中非常常见且危险的错误。

13. 一个真实科研问题如何从这套框架长出来

现在假设你观察到：任务描述稍微变得含糊后，Agent 更容易选错工具。不要立刻设计新方法。先按照标准科研流程把这个“现象”变成可验证问题。

科研元素	示例
Observation	任务含义相同，但表达更含糊时工具选择错误增加

科研元素	示例
Research Question	指令表述噪声是否系统性增加 tool-selection error？
Hypothesis	噪声增加会先影响 action selection，而非 tool execution
Independent Variable	指令噪声等级：0 / 1 / 2 / 3
Dependent Variables	success、wrong-tool rate、argument error、steps、latency
Controls	同模型、同任务语义、同工具集、同 evaluator、同预算
Trace evidence	对比 agent_action 与 tool_result 的错误位置

科研洞察 · 这一步非常关键

先观察现象，再提出机制假设，再设计实验。不要倒过来先发明一个“增强模块”，然后到处找数据证明它有效。

14. 从玩具 Benchmark 升级到可发表实验

本讲代码是“实验骨架”，不是最终论文。可发表研究通常需要逐层增加严谨性。最重要的是：每次增加复杂度，都知道它服务于哪个研究问题。

阶段	从本讲增加什么	目的
Stage 0	8 个 toy tasks + MockAgent	验证基础设施
Stage 1	真实 API + 50~100 个确定性任务	验证模型行为与 Trace
Stage 2	多模型 + 多 seed + 固定配置	检查普遍性与方差
Stage 3	研究变量：噪声 / 记忆 / 通信 / 角色 / 工具约束	验证机制假设
Stage 4	Baseline + Ablation + Failure taxonomy	排除替代解释
Stage 5	统计检验 + cost/latency + 公开代码/数据	形成可复现证据链

14.1 最低论文表格会长什么样？

System	Success ↑	Wrong Tool ↓	Arg Error ↓	Steps ↓	Cost ↓
Baseline Agent	0.xx	0.xx	0.xx	x.x	\$x.xx
+ Method / Condition A	0.xx	0.xx	0.xx	x.x	\$x.xx
+ Method / Condition B	0.xx	0.xx	0.xx	x.x	\$x.xx

科研洞察 · 先别急着刷 SOTA

对独立研究者更现实的路径，是找到一个稳定、可解释、可复现的系统现象，然后用 Trace 和受控实验解释其机制。只要问题本身重要，论文价值不必完全来自“总分超过所有人”。

15. 常见误区：这一讲最容易踩的 10 个坑

1. 把 Prompt 当成完整实验，没保存 task_id、配置和 Trace。
2. 一开始就上真实 API，基础设施 bug 和模型行为混在一起。
3. Agent、Harness、Evaluator 写在一个大函数里，无法控制变量。
4. 只保存最终答案，不保存 tool call、arguments 与中间结果。
5. 任务正确答案本身不可靠，却把模型判错。
6. 所有任务都用 LLM-as-a-Judge，导致评测器也随机。
7. 改方法时同时换模型、换 Prompt、换数据，无法归因。
8. 只跑一次，不报告随机 seed 与方差。
9. 看到 accuracy 变化就讲机制，没有 Trace 证据。
10. 为了“像 Agent”堆大量模块，反而让实验无法解释。

16. 动手实验：你现在应该完成的最小闭环

动手做 · 实验目标

不追求任何模型性能提升。只验证你已经具备独立运行 Agent 科研实验的最低闭环能力。

步骤	操作	你要确认的现象
1	打开 src/tasks.jsonl	每个任务有 task_id / category / instruction / expected
2	运行 mock benchmark	生成 outputs/ 三个文件
3	打开 results.jsonl	能找到 prediction / success / trace
4	故意把 calc_001 expected 改成 392	Evaluator 应报告失败
5	恢复 expected, 再把 calculator 工具名写错	Trace 应记录异常
6	恢复代码, 运行 analyze_results.py	能得到按 category 的统计

核心概念 · 为什么要故意制造错误？

因为你必须确认评测与 Trace 真能“抓住失败”。一个永远全对的 toy demo 很容易给人虚假的安全感；主动注入故障相当于给实验基础设施做单元测试。

17. 练习题

1. 为什么 Agent 和 Harness 必须分离？请用“控制变量”解释。
2. 如果 tool_result 是正确的，但 final_answer 错了，这属于哪一层 failure？为什么只看 accuracy 看不出来？
3. 什么时候应该优先用 deterministic evaluator，而不是 LLM-as-a-Judge？
4. 如果你想研究“多 Agent 错误相关性”，Trace 至少还要增加哪些字段？
5. 为什么 MockAgent 的 100% accuracy 不能证明这套 Agent 很强？
6. 把“任务描述噪声影响工具选择”改写为 Observation → RQ → Hypothesis → IV/DV。

18. 练习答案

1. Agent/Harness 分离：研究时我们经常只想替换“决策策略”，其他运行协议保持不变。若 Agent 与 Harness 混在一起，换一个方法往往同时改执行逻辑、错误处理、记录方式，导致无法判断效果来自哪里。
2. 这是 finalization / answer synthesis 层错误：证据已经正确，但模型在整合或格式化最终答案时破坏了正确结果。总 accuracy 只能告诉你失败，Trace 才能定位错误发生在工具之后。
3. 能规则化就优先规则化：数值、代码测试、结构化状态、集合相等任务应优先 deterministic evaluator。开放式文本质量、复杂偏好或难以程序化的语义判断才考虑 Judge，并要做 Judge 稳定性与偏差审计。
4. 至少增加 agent_id、round、message sender/receiver、message content/hash、shared observation、individual action、individual confidence、group decision。只有这样才能计算不同 Agent 的错误是否相关，以及错误如何通过通信传播。
5. MockAgent 是规则程序：它知道 task.category 并直接映射到工具，主要验证的是实验管道正确，而不是 LLM 的推理能力。

6. 示例：Observation：同义但更含糊的指令更易选错工具；RQ：噪声等级是否提高 wrong-tool rate？
Hypothesis：噪声首先破坏 action selection；IV：噪声等级；DV：wrong-tool rate、success、argument error、steps。

19. 本讲的最低闭环能力检查

- ☐ 我能解释 Task、Agent、Harness、Tool、Trace、Evaluator、Metrics 的职责。
- ☐ 我能先用 Mock 跑通，而不是一开始就把问题交给真实 API。
- ☐ 我知道怎样从 task_id 回到某次失败的完整 Trace。
- ☐ 我知道“最终失败”至少可以拆成工具选择、参数、工具执行、最终整合等不同 failure mode。
- ☐ 我能把一个观察到的失败现象改写成可实验的 RQ / Hypothesis / Variables。

核心概念 · 达到什么程度才算学会？

不是把代码背下来，而是你能独立解释：为什么每个模块存在、它隔离了什么变量、失败时去哪里查。这就是后续做真实 Agent benchmark、可靠性、通信、记忆、集体智能研究的基础。

20. 下一讲：把“跑通”升级成“可复现科研实验”

专题 A 第2讲将继续在这套工程上增加：Experiment Config、随机种子、重复运行、多模型对比、token/cost/latency 记录、失败类型 schema、结果目录版本化。目标是把一次 demo 变成可以支撑论文表格的可复现实验框架。

科研洞察 · 本讲一句话总结

Agent 科研的第一性问题不是“模型回答得多聪明”，而是“系统行为能不能被可靠地定义、执行、记录、评测和复现”。